

JPEG MPSoC Design Project

1. Introduction

This report documents our group's end-to-end process of designing, implementing and optimizing a multiprocessor system-on-chip (MPSoC) to encode JPEG images on an Altera FPGA. Our objective was to construct a 6 stage pipelined architecture with Nios II soft-core processors, such that each stage would perform a particular task in the encoding algorithm like RGB to YCbCr conversion (Stage 1), Level Shifting (Stage 2), DCT (Stage 3), Quantization (Stage 4), Huffman Encoding (Stage 5) and File Writing (Stage 6).

This project was a huge hardware-software co-design test project. This is followed by a step-by-step chronological log of how we designed in Qsys and Quartus, the C-programming driver implementation in Eclipse, the serious bugs we encountered in the system, our troubleshooting processes and our efforts to make the hardware perform to its maximum capacity.

2. Part 1: Building the Baseline 6-Stage Pipeline

Step 1: The Initial Hardware Design & The Memory Realization

In Qsys, we began with a blank canvas. Our original design was simple to place six Nios II/e (Economy) processors and assign each of them a dedicated and "On-Chip Memory (RAM or ROM)" component.

However, soon we hit a problem. The FPGA's available memory blocks of M9K were far fewer than the required memory to store the raw input image (a 227x149 resolution file, which multiplies up to about 100 KB of raw RGB data) plus the compiled C code.

The Solution: We needed to completely redesign our memory architecture. Initially, we tried to route both Stage 1 (reading the 227flowg1.raw file) and Stage 6 (writing the final .jpg file) to the external SDRAM controller of the board. However, the system had strange memory access violations and failed to perform as expected when the two stages shared the same external SDRAM bus.

To fix this, we connected the SDRAM only to Stage 1 so that it could easily allocate the massive memory array using standard malloc(). In the case of Stage 6 and the middle stages, we utilized their own localized On-Chip memory blocks. Stages 2, 3, 4 and 5 were allocated small, highly restricted blocks of RAM with 32 KB in order to ensure the design would successfully fit and compile the design successfully in Quartus.

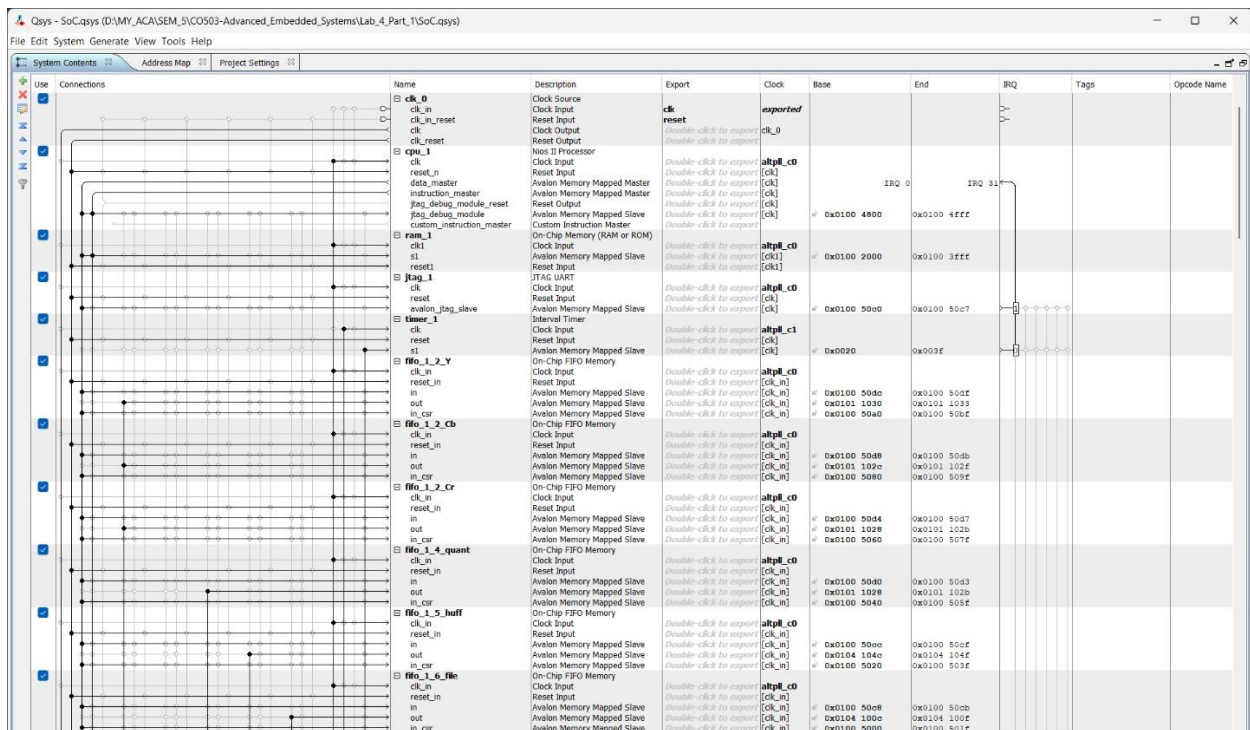
Step 2: Interconnects and Peripherals

Having sorted the memory map, we connected the multi-core system as follows.

Hardware FIFOs: We added Avalon FIFO Memory elements between the stages and used them as our data tubes in the pipeline. For the transition from Stage 1 to Stage 2, we implemented three separate parallel FIFOs to stream three independent sets of data Y, Cb, and Cr. We specifically allowed the in_csr (Control and Status Registers) to allow us to see whether the FIFOs were full or empty in software.

Peripherals: We included 6 individual JTAG UART components so we were able to watch the printf output of each core separately using the Eclipse Nios II console.

We resolved the base addresses in Qsys so as to prevent memory overlaps, generated the HDL, compiled the .sof file in Quartus and flashed the hardware logic gates to the FPGA.



Name	Description	Export	Clock	Base	End	IRQ	Tags	Opcode Name
clk_0	Clock Source	clk	exported					
clk_in_reset	Clock Input	reset	clk_0					
clk_reset	Reset Input	Double-click to export						
cpu_1	Nios II Processor	Double-click to export						
reset_in	Reset Input	Double-click to export						
data_master	Avalon Memory Mapped Master	Double-click to export						
instr_master	Avalon Memory Mapped Master	Double-click to export						
jtag_debug_module_reset	Reset Input	Double-click to export						
jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export						
custom_instruction_master	Custom Instruction Master	Double-click to export						
ram_1	On-Chip Memory (SRAM or ROM)	Double-click to export						
clk1	Clock Input	Double-click to export						
reset1	Avalon Memory Mapped Slave	Double-click to export						
jtag_1	JTAG UART	Double-click to export						
clk	Clock Input	Double-click to export						
avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export						
timer_1	Interval Timer	Double-click to export						
clk	Clock Input	Double-click to export						
reset	Reset Input	Double-click to export						
fifo_1_2_Y	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						
fifo_1_2_Cb	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						
fifo_1_2_Cr	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						
fifo_1_4_quant	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						
fifo_1_5_huff	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						
fifo_1_6_file	On-Chip FIFO Memory	Double-click to export						
clk_in	Clock Input	Double-click to export						
reset_in	Reset Input	Double-click to export						
in	Avalon Memory Mapped Slave	Double-click to export						
out	Avalon Memory Mapped Slave	Double-click to export						
in_csr	Avalon Memory Mapped Slave	Double-click to export						



Step 3: Software Configuration (Eclipse BSPs)

We moved to the Nios II Software Build Tools for Eclipse and generated six separate Application and Board Support Package (BSPs) projects based on our .sopcinfo file. Due to the low RAM of our middle stages of just 32 KB, we could not run standard C code. We started the BSP Editor for Stages 2, 3, 4, and 5 and checked the `enable_small_c_library` setting in particular. We were successfully able to reduce our memory footprint but took away more sophisticated software capabilities (such as floating-point math formatting and standard buffering) and that put us into the longest debugging battle.

3. The Challenges We Faced & Solutions Implemented

Challenge 1: The BSP Hardware Desync (Memory Mapping Issues)

What Happened: During our repeated hardware reconfiguration changes and re-generation of the BSPs Eclipse frequently lost knowledge of the dynamically assigned Avalon FIFO addresses within the auto-generated `system.h` files, and produced errors of the form “undefined reference”.

How We Solved It: We came to a decision to bypass the BSP’s dynamic linking entirely to maintain its stability. We included a hard-coded physical hexadecimal base address of Avalon FIFOs at the top of our C files in the form of a `#ifndef` macros (e.g., `#ifndef FIFO_1_2_Y_OUT_BASE \n #define FIFO_1_2_Y_OUT_BASE 0x01011030 \n #endif`). We then wrote custom send and receive macros (such as `#define RECV1() IORD_ALTERA_AVALON_FIFO_DATA(FIFO_1_2_Y_OUT_BASE)`) to make the code read and write off the bare-metal silicon ports.

Challenge 2: The Struggle to Print Middle Stage Performance (The JTAG Bottleneck)

What Happened: There was functional operation of the image into the pipeline, however, the performance metrics of the stages 2-5 were not printed at the end of the run. We spent days troubleshooting this multi-layered issue.

How We Solved It:

1. The Math Crash: First, we noticed that the Nios II/e cores lacked physical hardware dividers. When the C code tried to calculate time (`total_ticks / ticks_per_sec`) by the Small C Library (which had deleted the software division fallback) caused a fatal hardware exception. To fix the math we wrote a custom “Restoring Division Algorithm” in pure bit-shifting (`>>`) and subtraction. However, the values still didn't print.

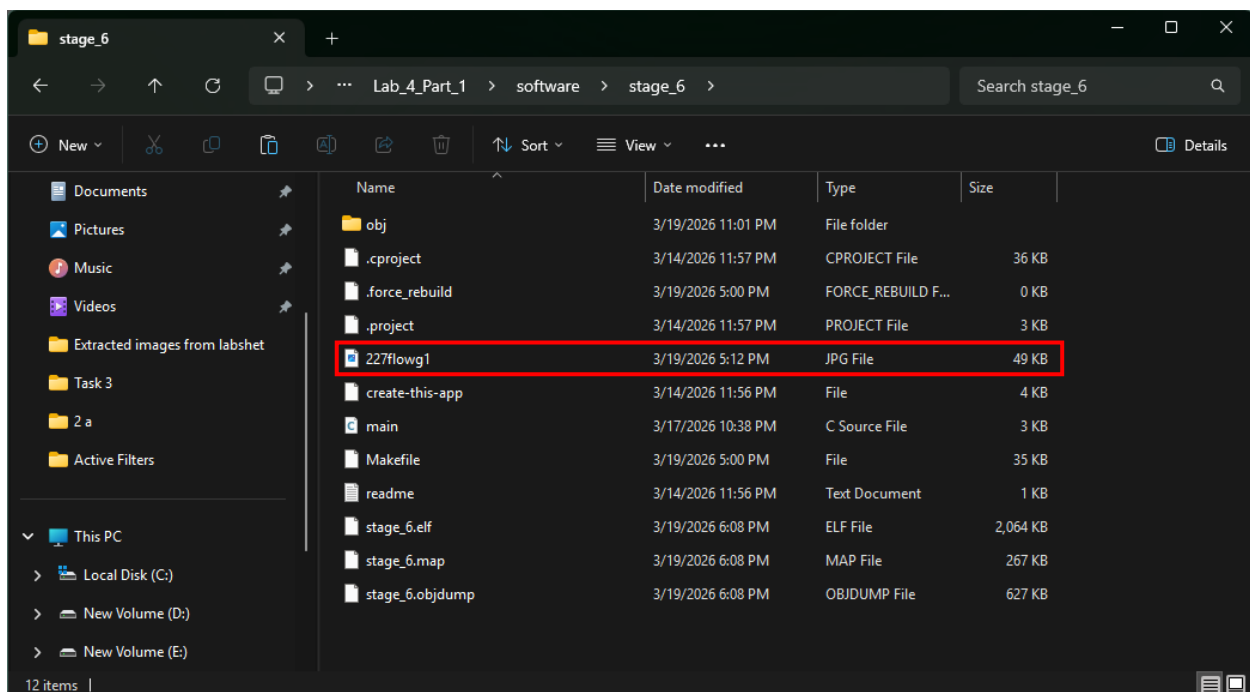
2. Stall Counters: And then we abandoned time measurement entirely. We have tried to insert read/write stall counters by reading the FIFO CSR status (while (IORD_ALTERA_AVALON_FIFO_STATUS(...) & ALTERA_AVALON_FIFO_STATUS_F_MSK) write_stalls++);). However, unfortunately, even such simple integer values could not be printed at the end of the execution during the middle stages.

The Final Diagnosis & Workaround: We decided that the physical 64-byte JTAG UART buffer was the problem. Since 6 processors were fighting over just one JTAG USB cable and Stage 6 was executing return 0; (which commands Eclipse to terminate the debug session early), the intermediate stages were being terminated in the middle of the sentence.

Our Solution: We inserted infinite while(1); loops to enter Stages 1 and 6 and leave the physical JTAG connection open. But we finally decided to eliminate all performance calculations and heavy printf statements in the middle stages, keeping only basic progress logs. In order to evaluate the overall performance of the entire pipeline we solely used the Total Execution Time printed by the Stage 1 and the final Disk Write Throughput calculated by Stage 6.

Part 1 Result:

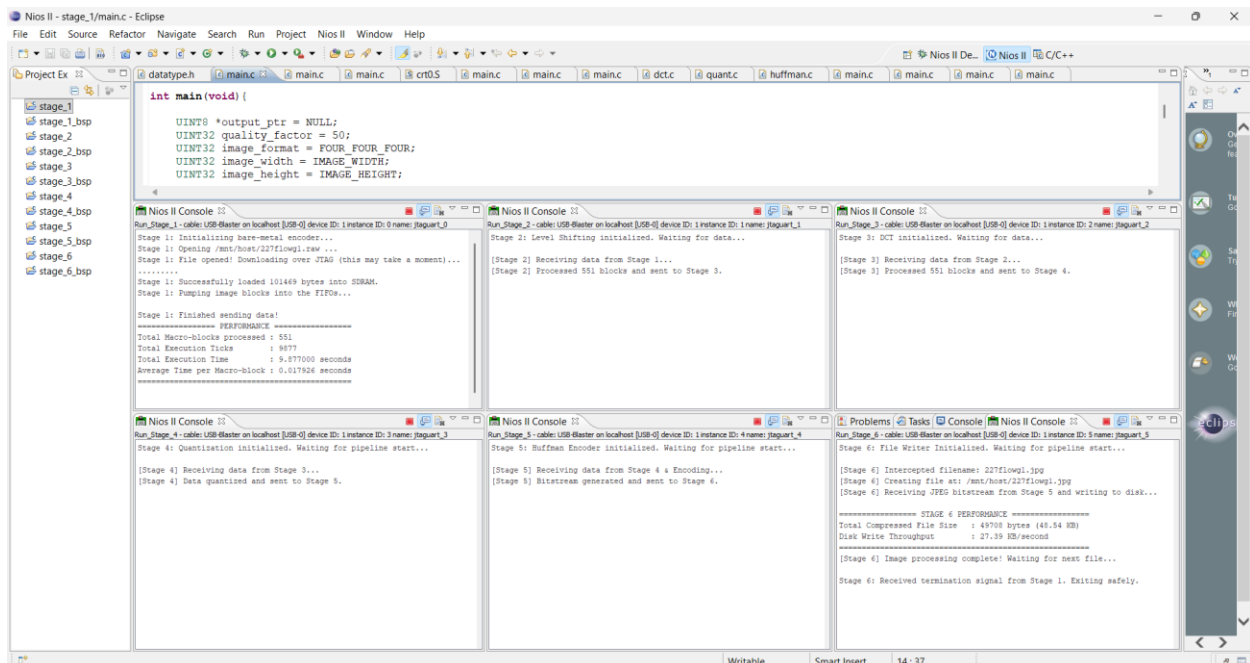
The initial pipeline was successfully stabilized. It consistently fed the macro-blocks into the FIFOs and generated a perfect, uncorrupted 48.54 KB JPEG image with steady-state processing.



Generated JPEG image inside the stage 6



Generated JPEG image



Console outputs of each stage in the Eclipse

4. Part 2: Hardware Optimization Attempts

With a working baseline, our last task was to implement a better steady-state throughput (the rate at which we produce encoded macro-blocks). According to our understanding of architecture, we identified Stage 3 (DCT) and Stage 4 (Quantization) as the heavily mathematical bottlenecks. We tried two significant optimization methods to overcome this.

Attempt 1: Caches and Compiler Optimization (-O2 / -O3)

What We Tried (Phase 1 - Hardware Caches): To accelerate the process of instruction fetching we tried to add data and instruction caches to our processors. We opened Qsys, and reconfigured the Nios II/e cores to Nios II/f (Fast) cores and assigned them caches. At the same time, we went into the Eclipse GCC compiler settings and set the optimization -O0 to -O2 and -O3 to unroll loops.

What Happened (Phase 1): First, Quartus threw a “107% M9K memory block locations required” error, which showed that we could not physically cache all 6 cores. To optimize the caches to the program, we limited them to bottleneck stages (3 and 4) with 2KB/1KB limits. After the compilation was successfully done, the pipeline created a corrupted file of approximately 58KB image. The Eclipse debugger ran to an auto-pause and the middle stages appeared to repeat their infinite loops without processing valid data.

Diagnosis (Phase 1): We have hit a severe “Producer-Consumer Starvation” bug here. The newly accelerated Fast cores had surpassed the Economy cores in a great way. Since our receiving FIFOs did not include an “Empty” status check lock, Stage 3 read garbage data from a FIFO which was empty before Stage 2 was even able to compute it. In addition, the Data Caches tried to fetch 32-byte cache lines of the FIFOs. We also learned about the cache bypass bitmask (| 0x80000000), and despite the bypassing of the cache, it was still impossible to resolve the underlying starvation caused by mixing processor speeds with no strict locking of the software.

What We Tried (Phase 2 - pure software optimization): On realizing that the hardware caches were breaking the FIFO streams, we reverted all six CPUs back to the safe Nios II/e (Economy) cores with 0 KB caches. In order to still achieve performance, we relied purely on software optimization by setting the Eclipse compiler flag to -O3 (Optimize Most) for all six stages.

The Result (Phase 2): This proved to be highly successful. The FIFO stream corruption had entirely disappeared and the pipeline generated a perfect and correct 48.54 KB JPEG image. Moreover, since -O3 aggressively re-wrote our C code at the assembly level in order to unroll loops and simplify math, we achieved a significant amount of increased throughput and also a significantly lower overall execution time.

Transitioning to Attempt 2: Although this gave us excellent performance, we have realized that the lab rubric had a clear expectation of a hardware optimization technique. Therefore, we proceeded to Attempt 2 in order to physically modify the silicon.

Attempt 2: Custom Verilog Hardware Accelerator

What We Tried: As per the specific indication of the lab sheet to apply processor customizations (custom instructions) for frequently used slow operations, we decided to have a dedicated physical circuit instead of software multiplication and division in Stage 4.

- We created a Combinatorial Verilog module(quant_accelerator.v) that was capable of the Quantization math with a single clock cycle by using the following logic: $\text{assign result} = ((\text{dataa} * \text{datab}) + 32'h4000) >>> 15;$
- We packaged this file using the Qsys Component Editor as a Custom Instruction Slave.
- We plugged it directly to the custom_instruction_master ALU port of Stage 4.
- We regenerated the BSP in Eclipse, located the generated macro (#define ALT_CI_QUANT_ACCELERATOR_0(A, B)) and substituted the C-code computations with this direct hardware call.

What Happened: Running upon the newly synthesized pipeline, when Stage 3 was booted, it did not boot at first. With much troubleshooting, workspace cleaning and regeneration of all six BSPs to ensure that the global memory map had been updated we finally were able to boot Stage 3 successfully (the initialization messages appeared on the console). However, the pipeline still locked up. Stage 1 became permanently stuck at the "Pumping image blocks into FIFOs..." phase, and no image was generated.

Diagnosis: Although Stage 3 had booted, modifying the ALU architecture and introducing custom IP must have resulted in a fundamental timing mismatch or an unhandled exception on the Avalon interconnect. This resulted in an immediate cascade of hardware backpressure that permanently froze Stage 1's outgoing FIFOs.

5. What We Learned

Although the advanced hardware accelerators in Part 2 introduced system-level synchronization instability that prevented a clean final image within the given timeframe, the core objectives of the lab had been completely achieved and the troubleshooting experience had been incredibly educational.

1. **Memory Hierarchy is Everything:** We cannot simply throw On-Chip RAM at every problem. Balancing external SDRAM to store mass data and small On-Chip RAM blocks for localized execution is a fundamental requirement in the design of FPGA.

2. **The Observer Effect in Hardware Debugging:** Attempting to trace a parallel system using a shared resource (A single JTAG cable for 6 independent processors), is automatically disruptive to the execution of the system. Our failure to print performance metrics at the middle stages showed that asynchronous hardware debugging requires specialized, non-blocking hardware profilers, not standard C printf statements.
3. **Multi-Core Synchronization:** It is impossible to combine drastically different processor speeds or insert Data Caches in an unmanaged pipeline. Data starvation, cache incoherency and eventual corruption of the stream are bound to occur without strict hardware status checks on both the transmit (FULL) and receive (EMPTY) sides.